ShadowShare

FRONTEND SPECIFICATION · V2.0

The trusted zone.

A complete specification for the React/TypeScript browser application that performs all encryption, decryption, and key handling for ShadowShare.

FRAMEWORK React 18 + Vite 5	LANGUAGE TypeScript 5.4 (strict)
STYLING Tailwind CSS 3.4	CRYPTO Web Crypto API (native)
STATE Zustand 4.5	ROUTING React Router 6.22

TypeScript Strict · Tailwind · Web Crypto · WCAG 2.2 AA

TABLE OF CONTENTS

01 Frontend Mission & Scope

The ShadowShare frontend is the **only place** where user plaintext, encryption keys, and access passwords ever exist. It is therefore not merely a UI layer — it is the entire trusted compute zone of the product. Every architectural decision (no third-party scripts, no service worker, no analytics, strict CSP, type-safe crypto interfaces) flows from this responsibility.

The frontend has four primary responsibilities:

1. **Cryptographic execution** — PBKDF2 key derivation, AES-256-GCM encryption / decryption, IV generation, auth-tag verification.
2. **Key transport** — embed key material in the URL fragment for the sender; read it back from `location.hash` for the recipient.
3. **UX state machine** — drive the sender and recipient through encrypt / upload / share / decrypt / download phases.
4. **Trust communication** — make the crypto guarantees visible, comprehensible, and verifiable to both technical and non-technical users.

> **SINGLE INVIOLABLE RULE**
>
> Nothing that crosses the boundary out of the browser may contain plaintext, the encryption key, the user's password, or any pre-image of either. This rule is enforced through code review, unit tests, and an ESLint rule that flags suspicious `fetch` bodies.

02 Tooling & Build Pipeline

TOOL	VERSION	ROLE
Vite	5.x	Dev server (HMR), production build (esbuild + Rollup)
TypeScript	5.4	Strict mode; no `any`; `noUncheckedIndexedAccess`
React	18.3	UI framework
React Router	6.22	Client routing
Zustand	4.5	App state (no Redux required)
Tailwind CSS	3.4	Atomic CSS; design tokens via theme config
Phosphor Icons	2.x (React, duotone)	Single icon library, 20/24/40/64 sizes
JSZip	3.10	F7 multi-file bundling, client-side
qrcode	1.5	F9 QR rendering, client-side
pdfjs-dist	4.x	F8 PDF page-1 preview
Vitest	1.x	Unit / integration test runner
Playwright	1.x	End-to-end browser tests
ESLint	9.x	Lint + custom security rules
Prettier	3.x	Formatting (single-quote, 2 spaces)

2.1 Build Output

- Single SPA bundle with code-splitting on the viewer route (PDF.js loaded on demand).

- Subresource integrity (SRI) hashes generated by Vite plugin for every emitted JS/CSS asset.
- Source maps emitted but uploaded only to private Sentry — not served publicly.
- No CDN dependencies in the critical path (no Google Fonts on production — Inter and JetBrains Mono are self-hosted and preloaded).

03 Project Structure

```
shadowshare-web/
├── public/
│   ├── favicon.svg
│   └── fonts/                  # self-hosted Inter, JetBrains Mono
├── src/
│   ├── main.tsx                # React entrypoint
│   ├── App.tsx                 # Router shell, theme provider
│   ├── routes/
│   │   ├── UploadPage.tsx
│   │   ├── ViewerPage.tsx
│   │   ├── StatusPage.tsx
│   │   ├── DeletePage.tsx
│   │   ├── NotFoundPage.tsx
│   │   └── UnsupportedBrowserPage.tsx
│   ├── components/
│   │   ├── ui/                 # 12-component library (see §9)
│   │   ├── upload/             # DropZone, SecurityModeSelector, AdvancedOptions
│   │   ├── viewer/             # IntegrityBadge, PreviewPane, Countdown
│   │   ├── share/              # ShareLinkBox, QRPopover, DeleteLinkBox
│   │   └── modals/             # HowItWorksModal, ConfirmModal
│   ├── crypto/
│   │   ├── kdf.ts              # PBKDF2 wrappers
│   │   ├── aead.ts             # AES-GCM encrypt/decrypt
│   │   ├── random.ts           # IV + key generation; uniqueness asserts
│   │   ├── fragment.ts         # URL fragment encode/decode
│   │   ├── verifier.ts         # access-password verifier (F5)
│   │   └── index.ts            # public API of crypto module
│   ├── api/
│   │   ├── client.ts           # typed fetch wrapper
│   │   ├── upload.ts           # chunked upload orchestrator (F10)
│   │   ├── download.ts         # streaming download + tamper detection
│   │   └── status.ts           # sender status fetcher
│   ├── state/
│   │   ├── uploadStore.ts      # Zustand: phase, progress, file list
│   │   ├── viewerStore.ts      # Zustand: decryption state, preview
│   │   └── themeStore.ts       # Zustand: theme (system/dark/light)
│   ├── hooks/
│   │   ├── useDropzone.ts
│   │   ├── useCountdown.ts
│   │   ├── useClipboard.ts
│   │   └── useTheme.ts
│   ├── lib/
│   │   ├── zip.ts              # JSZip wrapper for F7
│   │   ├── preview.ts          # F8 preview helpers (img + PDF.js)
```

```
│   │   ├── qrcode.ts           # F9 QR helper
│   │   ├── format.ts           # bytes, dates, durations
│   │   └── browserSupport.ts   # capability detection
│   ├── styles/
│   │   ├── tokens.css          # design tokens (light + dark)
│   │   └── globals.css
│   └── types/
│       └── api.ts              # API DTOs (typed with zod schemas)
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── index.html
├── vite.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── eslint.config.js
└── package.json
```

04 Routing

PATH	COMPONENT	PURPOSE
`/`	UploadPage	Sender uploads a file
`/v/:fileId`	ViewerPage	Recipient downloads a file. Key in `#` fragment.
`/status/:fileId`	StatusPage	Sender views status. Delete token in `#`.
`/delete/:fileId`	DeletePage	Sender kill switch. Delete token in `#`.
`/unsupported`	UnsupportedBrowserPage	Fallback for missing Web Crypto / WebKit too old.
`*`	NotFoundPage	404

All non-root paths read sensitive material from `location.hash` in `useEffect` and immediately copy it to in-memory React state; the URL is then optionally rewritten via `history.replaceState` to remove the fragment from the address bar (configurable behaviour).

05 State Management

Three independent Zustand stores, intentionally narrow in scope.

5.1 uploadStore

```
type UploadPhase =
  | 'idle' | 'bundling' | 'deriving-key' | 'encrypting'
  | 'uploading' | 'success' | 'error';

interface UploadState {
  files: File[];                 // selected by user
  securityMode: 'link+password' | 'link-only';
  password: string;              // never persisted
  accessPassword: string | null; // F5
  ttl: '1h' | '24h' | '7d' | '30d';
  burnAfterRead: boolean;
  maxDownloads: number | null;
  phase: UploadPhase;
  progressPct: number;
  result: { fileId: string; key: string; deleteToken: string } | null;
  error: string | null;
  // actions
  addFiles(files: File[]): void;
  removeFile(index: number): void;
  setSecurityMode(mode): void;
  setPassword(p: string): void;
  /* ... */
  startUpload(): Promise<void>;
  reset(): void;
}
```

5.2 viewerStore

```
type ViewerPhase =
  | 'loading' | 'awaiting-access-password' | 'awaiting-password'
  | 'decrypting' | 'ready' | 'tampered' | 'expired' | 'not-found' | 'error';

interface ViewerState {
  fileId: string;
  flags: { burnAfterRead: boolean; hasAccessPassword: boolean; expiresAt: string };
  phase: ViewerPhase;
  decryptedBlob: Blob | null;
  decryptedMeta: { filename: string; mime: string; size: number } | null;
  previewUrl: string | null;
  error: string | null;
  /* actions */
  init(fileId: string, fragment: string): Promise<void>;
  submitAccessPassword(p: string): Promise<void>;
  submitPassword(p: string): Promise<void>;
  download(): void;
  reset(): void;
}
```

5.3 themeStore

```
type Theme = 'system' | 'dark' | 'light';
interface ThemeState {
  theme: Theme;
  resolvedTheme: 'dark' | 'light';
  setTheme(t: Theme): void;
}
```

> **MEMORY HYGIENE**
>
> Both `uploadStore.password` and `viewerStore.decryptedBlob` hold sensitive data. They are **cleared** on route change and on `beforeunload`. They are never persisted to localStorage, sessionStorage, IndexedDB, or cookies. Devtools middleware is disabled in production builds (`import.meta.env.PROD`).

06 Cryptography Module

All cryptography lives in `src/crypto/`. Nothing outside this directory calls `crypto.subtle` directly; this enables a single audit surface.

6.1 Public API

```
// src/crypto/index.ts

export interface DerivedKey { key: CryptoKey; salt: Uint8Array; }

export async function deriveKeyFromPassword(
  password: string,
  salt?: Uint8Array,                    // optional — undefined ⇒ generate fresh
): Promise<DerivedKey>;

export async function generateRandomKey(): Promise<{ key: CryptoKey; raw: Uint8Array }>;

export async function encryptBytes(
  key: CryptoKey,
  plaintext: Uint8Array,
): Promise<{ ciphertext: Uint8Array; iv: Uint8Array }>;

export async function decryptBytes(
  key: CryptoKey,
  ciphertext: Uint8Array,
  iv: Uint8Array,
): Promise<Uint8Array>;   // throws on auth-tag failure

export function encodeShareFragment(opts: {
  mode: 'pwd' | 'key';
  salt?: Uint8Array;
  rawKey?: Uint8Array;
  flags: { burn: boolean; hasAccessPwd: boolean };
}): string;

export function decodeShareFragment(fragment: string): {
  mode: 'pwd' | 'key';
  salt?: Uint8Array;
  rawKey?: Uint8Array;
  flags: { burn: boolean; hasAccessPwd: boolean };
};

export async function buildAccessVerifier(accessPassword: string):
  Promise<{ verifier: Uint8Array; salt: Uint8Array; iv: Uint8Array }>;

export async function checkAccessVerifier(
  accessPassword: string,
  verifier: Uint8Array, salt: Uint8Array, iv: Uint8Array,
): Promise<boolean>;
```

6.2 PBKDF2 (kdf.ts)

```
const PBKDF2_ITERATIONS = 100_000;
const SALT_BYTES = 16;
const KEY_BYTES = 32;

export async function deriveKeyFromPassword(password, salt) {
  const realSalt = salt ?? crypto.getRandomValues(new Uint8Array(SALT_BYTES));
  const baseKey = await crypto.subtle.importKey(
    'raw', new TextEncoder().encode(password),
    { name: 'PBKDF2' }, false, ['deriveKey']
  );
  const key = await crypto.subtle.deriveKey(
    { name: 'PBKDF2', salt: realSalt, iterations: PBKDF2_ITERATIONS, hash: 'SHA-256' },
    baseKey,
    { name: 'AES-GCM', length: 256 },
    /* extractable */ false, ['encrypt', 'decrypt']
  );
  return { key, salt: realSalt };
}
```

6.3 AES-256-GCM (aead.ts)

```
const IV_BYTES = 12;

export async function encryptBytes(key, plaintext) {
  const iv = crypto.getRandomValues(new Uint8Array(IV_BYTES));
  const ct = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, key, plaintext);
  return { ciphertext: new Uint8Array(ct), iv };
}

export async function decryptBytes(key, ciphertext, iv) {
  // Web Crypto throws on auth-tag mismatch — propagate as TamperedError
  try {
    const pt = await crypto.subtle.decrypt({ name: 'AES-GCM', iv }, key, ciphertext);
    return new Uint8Array(pt);
  } catch (e) {
    throw new TamperedError('AEAD authentication failed', { cause: e });
  }
}
```

6.4 IV uniqueness enforcement

A unit test in `tests/unit/crypto.iv.test.ts` generates 1,000,000 IVs and asserts zero collisions. A runtime `WeakSet` in dev builds also flags any IV reused within a single tab session (developer aid only — not present in production).

6.5 Share fragment format

```
// Link + Password mode (F5 capable)
#{base64url(salt)}.{flagsByte}

// Link Only mode (F6)
#k.{base64url(rawKey)}.{flagsByte}

// flagsByte bits:
//   bit 0 — burn_after_read
//   bit 1 — has_access_password
//   bits 2-7 — reserved (zero)
```

07 API Client Layer

`src/api/client.ts` is a 60-line typed `fetch` wrapper. It provides:

- Automatic `Content-Type` and `Accept` headers.
- Zod-validated response parsing — every DTO has a schema in `src/types/api.ts`.
- Typed error union: `NetworkError | NotFoundError | ExpiredError | RateLimitError | ServerError`.
- Strict policy: **never** includes `location.hash`, the password, or the key in any request body, query string, or header.

7.1 Chunked Upload Orchestrator (F10)

```
// src/api/upload.ts (pseudocode)
const CHUNK_SIZE = 5 * 1024 * 1024;

async function uploadChunked(blob: Uint8Array, settings, onProgress) {
  const { sessionId } = await api.post('/api/upload/init', {
    total_size: blob.length,
    total_chunks: Math.ceil(blob.length / CHUNK_SIZE),
    settings,
  });

  const status = await api.get(`/api/upload/status/${sessionId}`);
  let start = status.lastChunkIndex + 1;

  for (let i = start; i < totalChunks; i++) {
    const slice = blob.subarray(i * CHUNK_SIZE, (i + 1) * CHUNK_SIZE);
    await retryWithBackoff(() ⇒
      api.put(`/api/upload/chunk/${sessionId}/${i}`, slice)
    );
    onProgress((i + 1) / totalChunks);
  }

  return api.post(`/api/upload/complete/${sessionId}`, { settings });
}
```

08 Design System & Tokens

8.1 Color Tokens (Dark mode primary)

TOKEN	DARK	LIGHT	USE
`--bg`	#0A0C12	#F7F8FA	App background
`--surface`	#12151F	#FFFFFF	Card / panel
`--elevated`	#1C2030	#EFF1F6	Inputs, hover
`--border`	#252A3A	#E2E5EE	Borders
`--brand`	#7C6EFA	#5B4DEB	Primary accent
`--brand-hover`	#9B8FFB	#7C6EFA	Hover accent
`--brand-dim`	#2A2560	#EEEBFF	Tinted backgrounds
`--text`	#EEF0F6	#0A0C12	Primary text
`--text-2`	#8B92A8	#3C4255	Secondary text
`--text-3`	#525869	#6B7185	Muted / placeholder
`--success`	#27D585	#1AB873	Success / verified
`--warning`	#F5A623	#D88A0F	Burn / expiry
`--danger`	#EF5350	#D63F3C	Tamper / error

8.2 Typography

- **Primary:** Inter (400 / 500 / 600 — never 700+ except hero display).
- **Monospace:** JetBrains Mono — used for share links, file IDs, technical footnotes.
- **Scale (px):** 12 / 13 / 14 / 16 / 18 / 24 / 32 / 48.

- **Line-height:** 1.5 body, 1.2 headings.

8.3 Spacing, Radius, Motion

- **Base unit:** 4px. All margins/paddings are multiples of 4.
- **Radius:** 6 (buttons/inputs) · 10 (cards) · 16 (large panels) · 24 (hero card) · 9999 (pills).
- **Motion:** max 350 ms; cubic-bezier(0.4, 0, 0.2, 1) by default; reduced-motion media query disables non-essential animation.

09 Component Library (12 Components)

All components live in `src/components/ui/`. Each has: (a) a TypeScript prop interface, (b) every documented state, (c) Storybook story, (d) Vitest unit test, (e) accessibility attributes.

9.1 Button

Variants: `primary` · `outline` · `ghost` · `danger`. Sizes: `sm` · `md` · `lg`. States: default · hover · active · focus · disabled · loading.

```
<Button variant="primary" size="lg" loading={isEncrypting}>
  Encrypt & Upload
</Button>
```

9.2 Input

Variants: `text` · `password` (with show/hide eye) · `number`. States: default · focus · error · disabled. ARIA: `aria-invalid`, `aria-describedby` for helper / error.

9.3 DropZone

States: `idle` · `drag-hover` · `rejected` · `file-selected` · `disabled`. Visual transitions per design spec (dashed→solid border, scaled icon).

9.4 TogglePill

Two-or-more-option segmented control. Used for security mode (F6) and TTL (F2).

```
<TogglePill
  value={ttl}
  onChange={setTtl}
  options={[{value:'1h',label:'1h'},{value:'24h',label:'24h'},
            {value:'7d',label:'7d'},{value:'30d',label:'30d'}]}
/>
```

9.5 ToggleSwitch

Binary on/off. Used for burn-after-read (F1). 32 px track, animated thumb, focus ring.

9.6 ProgressBar

Three-phase: *deriving key → encrypting → uploading*. Shimmer overlay while progress < 100%. ARIA `role="progressbar"` with `aria-valuemin/max/now`.

9.7 ShareLink

Read-only input + copy button + QR popover (F9). Copy state animates to checkmark for 2 s. Click anywhere on the input selects all.

9.8 Toast

Four variants: success / error / warning / info. Auto-dismiss after 4 s; pause on hover; dismissible. Stacked top-right on desktop, bottom-centred on mobile.

9.9 Modal

Backdrop `bg-black/70 backdrop-blur-sm`. Focus trap. ESC to close. On mobile (< 640px) it transforms to a bottom sheet with `rounded-t-2xl` and slide-up animation.

9.10 Badge / Chip

Variants: neutral · brand · success · warn · danger · new. Used for trust indicators ("AES-256-GCM"), file status, "Integrity verified".

9.11 FileTypeIcon

Colour-coded by MIME family: image (blue) · doc (purple) · zip (amber) · pdf (red) · video (green) · audio (pink) · text (slate) · generic (gray). 8+ types.

9.12 Spinner

Four sizes (16 / 20 / 32 / 40). Brand-coloured. Stroke-only style.

10 Screen Specifications

10.1 Upload Page — Idle State

Full viewport, dark background `#0A0C12`. Subtle 3% noise overlay. Centred column, max width **560 px**.

Header (fixed, 60 px)

- Left: logo (28 px) + wordmark.
- Right: *"How it works"* link + theme toggle icon button.
- Background transparent → surface on scroll. Bottom border 0.5 px `--border`.

Hero

- Pill badge: *"Open Source · Zero Knowledge · Auditable"* — brand-dim background, 12 px Inter.
- Headline (48 px Inter 600): *"Send files." / "No trace."* Second line gets `letter-spacing: 0.02em`.
- Subheadline (16 px `--text-2`): two-line trust statement.
- Trust badges row: *AES-256-GCM*, *Zero-Knowledge*, *Client-Side Only*.

Main Upload Card (glassmorphism)

Single glassmorphism surface: `background: rgba(18,21,31,0.85); backdrop-filter: blur(24px); border: 1px solid rgba(255,255,255,0.06);` Padding 32 px, radius 24 px.

Sections in order:

1. **A — DropZone** — three visual states (idle, drag-hover, file-selected).
2. **B — Security Mode Selector** (F6) — TogglePill with Link+Password vs Link-Only.
3. **C — Password Input** — visible only in F5 mode. Eye toggle, helper text, character dot indicator.
4. **D — Advanced Options** — collapsible. Contains TTL segmented control (F2), burn-after-read toggle (F1), optional max-downloads input.
5. **E — Encrypt & Upload button** — primary, 48 px, full width.

10.2 Upload Page — Progress State

Card content fades out; progress UI fades in. Three sequential phases each with its own animated icon (rotating key → pulsing lock → floating cloud-upload), label (Inter 600 18 px), and sublabel (Inter 13 px monospace technical detail). Progress bar with shimmer overlay. Cancel link visible only during upload phase.

10.3 Upload Page — Success State

- Animated stroke-only check (400 ms circle draw + 200 ms tick draw) followed by a one-shot 600 ms confetti burst.
- Heading: *"File encrypted & ready"*.
- ShareLink box (component 9.7) with QR popover.
- Expiry chip: *clock icon + "Expires in 7 days"*.
- Warning callout: *"This link contains the decryption key. Share it only via a private channel — do not post publicly."*
- **Delete-link box (F3)** — separate row, less prominent than share link, with tooltip explaining the kill switch.
- *"Send another file"* link.

10.4 Viewer Page

State machine with six terminal states. Same dark background, centred 480 px card.

STATE	TRIGGER	UI
Loading	Initial mount	Spinner + *"Preparing secure transfer..."*
Access Password Required	F5 flag set	Lock icon, password input, *"The sender protected this file."*
Password Required	F5 mode (encryption pwd)	Password input + *"Enter the password the sender gave you."*
Decrypting	After password	Animated padlock opens; *"Checking AES-GCM authentication tag"*
Ready	Auth tag verified	File icon, filename, integrity badge, optional preview, download button, optional burn warning, countdown
Tampered	AEAD failure	Shield-X 64 px `--danger`, *"Integrity check failed"*, no download button
Expired	HTTP 410	Clock 64 px `--warning`, *"This file has expired"*, "Send a file" CTA
Not Found	HTTP 404	Generic not-found, *"This link is no longer valid."*

10.5 Status Page (F4)

Accessed via `/status/:fileId#deleteToken`. Card shows:

- File status pill: *Active* · *Expired* · *Deleted*
- Upload time (humanised: "uploaded 12 minutes ago")
- Download count (large number, brand-coloured)
- First access time (or *"Not yet opened"*)
- Time remaining (live countdown)
- Burn-after-read indicator
- Big red **"Delete now"** button (kill switch)
- Auto-refresh every 30 s; visible "refreshing..." indicator

10.6 "How It Works" Modal (F11)

Three vertical animated steps (browser encrypts → server stores noise → key in fragment). Each with a primary icon, a paragraph for non-technical readers, and a monospace footnote for technical readers ("PBKDF2 · 100,000 iterations · SHA-256"). Bottom links: *Open source*, *Read the security audit*.

10.7 404 & Unsupported Browser Pages

404: large brand-coloured "404", *"Nothing here."*, animated subtle SVG grain, "Go home" link.
Unsupported: alert-triangle, list of minimum browser versions (Chrome 90+, Firefox 92+, Safari 15+, Edge 90+), banner if served over HTTP.

11 Feature Implementation Details

F1 — Burn-After-Reading (frontend role)

Setting is sent as a public flag in upload settings. The viewer page reads the flag via the metadata endpoint and renders the amber warning callout above the download button. No additional frontend logic; deletion is purely server-driven.

F2 — Custom TTL

TogglePill component (4 options). On viewer side, `useCountdown(expiresAt)` hook drives a 1 Hz timer; falls back to *"Expires in < 1 minute"* within the final 60 seconds; transitions to *Expired* state when the timer hits zero **and** the next backend call returns 410.

F3 — Sender Kill Switch

Upload result includes `deleteToken`. Frontend builds two URLs:

```
const shareUrl   = `${origin}/v/${fileId}#${fragment}`;
const deleteUrl  = `${origin}/delete/${fileId}#${deleteToken}`;
const statusUrl  = `${origin}/status/${fileId}#${deleteToken}`;
```

Delete page reads token from `location.hash` and on confirmation calls `DELETE /api/file/{id}?token=…`.

F4 — Download-Count Status

StatusPage polls `GET /api/status/{id}?token={deleteToken}` every 30 s with `setInterval`; cancelled on unmount. No personal data is fetched or rendered.

F5 — Access Password

At upload, after the sender enters an access password, the client:

```
const { verifier, salt, iv } = await crypto.buildAccessVerifier(accessPwd);
// verifier is uploaded to server as opaque bytes
```

At viewer time, if `flags.hasAccessPassword` is true, the UI prompts for the access password and:

```
const ok = await crypto.checkAccessVerifier(input, verifier, salt, iv);
if (!ok) setError('Incorrect password');
```

Critically, this round-trip never includes the password — the server cannot brute-force it because it does not see attempts.

F6 — Link-Only Mode

When selected, the client generates a fresh 256-bit AES-GCM key via `crypto.subtle.generateKey`, exports the raw bytes, base64url-encodes them and writes them into the fragment as `k.{rawKey}`. The recipient's viewer detects the `k.` prefix and skips the password prompt.

F7 — Multi-File / Folder Upload

`src/lib/zip.ts` wraps JSZip:

```
export async function bundleFiles(files: File[]): Promise<Uint8Array> {
  const zip = new JSZip();
  for (const f of files) zip.file(f.webkitRelativePath || f.name, f);
  return new Uint8Array(await zip.generateAsync({ type: 'arraybuffer' }));
}
```

Bundled bytes feed the same encryption pipeline. The encrypted-metadata blob marks the content as `application/zip` with a synthetic filename (e.g. `shadowshare-bundle-2026-05-28.zip`).

F8 — Client-Side Preview

After successful decryption, `src/lib/preview.ts` inspects MIME:

- `image/*` → `URL.createObjectURL(new Blob([bytes], {type: mime}))` → `<img>` rendered into a contained `max-h-60` area. `URL.revokeObjectURL` on unmount.
- `application/pdf` → PDF.js renders page 1 only, into a canvas, from the in-memory Uint8Array.
- Other MIME → no preview; *"No preview available — click Download"*.

F9 — QR Code

QR rendered into 200×200 canvas using the `qrcode` package with options:

```
{ errorCorrectionLevel: 'M',
  margin: 1, scale: 6,
  color: { dark: '#7C6EFA', light: '#0A0C12' } }
```

F10 — Chunked Resumable Upload

See §7.1. Chunk failures are retried with exponential backoff (1s, 2s, 4s, 8s, max 3 retries) before surfacing to the user. Progress reported to `uploadStore` at chunk granularity.

F11 — How It Works modal

Implemented as a portal rendered via the Modal component. Animations are SVG path stroke animations; respects `prefers-reduced-motion`.

F12 — Theme system

See §12 for full details.

F13 — Security audit artefact

Not a frontend feature directly; the frontend exposes the *"Read the security audit"* link in the How It Works modal and in the footer, pointing to the `SECURITY.md` in the public GitHub repository.

12 Theme & Dark/Light Mode

Implementation lives in `src/state/themeStore.ts` and `src/hooks/useTheme.ts`.

```
// On first load (before React paints) — inline script in index.html:
<script>
  (function(){
    var s = localStorage.getItem('shadowshare-theme') || 'system';
    var resolved = s === 'system'
      ? (matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light')
      : s;
    document.documentElement.dataset.theme = resolved;
  })();
</script>
```

This inline blocking script prevents flash-of-wrong-theme. The Zustand store hydrates from `localStorage` after mount and stays in sync with `matchMedia` changes when `theme === 'system'`.

Tailwind config:

```
// tailwind.config.ts
darkMode: ['class', '[data-theme="dark"]']
```

13 Accessibility

Target: **WCAG 2.2 AA**. Lighthouse accessibility score ≥ 95.

- All interactive elements are reachable by keyboard; visible focus ring (`ring-2 ring-brand/60`).
- All form fields have associated `<label>`; errors announced via `aria-live="polite"`.
- The drop zone is a button (`role="button"`) and accepts `Enter` / `Space` to open the file picker.
- Progress bar uses `role="progressbar"` with min/max/now.
- Modal traps focus, restores focus to opener on close, closes on Escape.
- Colour is never the sole signal — every state has icon + text + colour.
- `prefers-reduced-motion` disables non-essential animations (confetti, padlock opening, progress shimmer).
- Minimum touch target 44 × 44 px on mobile.
- Language declared in `<html lang="en">`.

14 Performance Budget

METRIC	BUDGET	ENFORCEMENT
Initial JS (gzipped)	≤ 180 KB	Vite bundle-analyzer; CI fails over budget
Initial CSS (gzipped)	≤ 20 KB	Tailwind purge + CI check
Total transfer for upload page	≤ 250 KB	Lighthouse
Time to Interactive (cold, fast 4G)	≤ 1.5 s	Lighthouse
LCP	≤ 1.8 s	Lighthouse
CLS	≤ 0.05	Lighthouse
Encryption throughput	≥ 200 MB/s	Manual benchmark, recorded in audit
PDF.js loaded only on viewer route	Code-split	Vite manualChunks

15 Browser Support

BROWSER	MINIMUM	RATIONALE
Chrome / Chromium	90+	Web Crypto AES-GCM stable, modern ES, top-level await
Firefox	92+	Web Crypto, modern ES
Safari	15+	Web Crypto stable; `webkitdirectory` works in 15.4+
Edge (Chromium)	90+	Same as Chrome
iOS Safari	15.4+	Web Crypto + folder uploads
Internet Explorer	Not supported	No Web Crypto AES-GCM; redirected to `/unsupported`

Capability check in `src/lib/browserSupport.ts`:

```
export function isSupported() {
  if (!window.crypto?.subtle) return false;
  if (typeof window.crypto.getRandomValues !== 'function') return false;
  if (typeof structuredClone !== 'function') return false;
  return true;
}
```

16 Frontend Security Guarantees

16.1 What the frontend MUST do

- Encrypt all file bytes locally before any network call.
- Generate IVs via `crypto.getRandomValues` with proven uniqueness.
- Verify the GCM auth tag before exposing decrypted bytes to the UI.

- Strip the URL fragment from any Sentry breadcrumb / error report.
- Clear sensitive state on route change and on `beforeunload`.

16.2 What the frontend MUST NOT do

- Send the password, key, IV, or any plaintext byte to the server.
- Register a service worker (would intercept fetches with fragment in scope).
- Load any third-party JavaScript in the critical path (no analytics, no chat widget, no fonts CDN).
- Store sensitive data in localStorage / sessionStorage / IndexedDB / cookies.
- Use `eval`, `Function` constructor, or `dangerouslySetInnerHTML` — ESLint rules enforce.

16.3 Content Security Policy (frontend-facing)

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';            // tailwind generates inline; reviewed
  font-src 'self';
  img-src 'self' data: blob:;                  // blob: required for in-memory previews
  connect-src 'self';
  frame-ancestors 'none';
  base-uri 'self';
  form-action 'none';
  upgrade-insecure-requests;
```

17 Testing

17.1 Unit Tests (Vitest)

- `crypto/kdf.test.ts` — PBKDF2 against RFC 6070 vectors.
- `crypto/aead.test.ts` — AES-GCM against NIST CAVP vectors.
- `crypto/iv.test.ts` — 1,000,000 IV uniqueness.

- `crypto/fragment.test.ts` — round-trip encode/decode for both modes.
- `crypto/verifier.test.ts` — wrong password rejected, right accepted.
- `state/uploadStore.test.ts` — state machine transitions.
- `lib/zip.test.ts` — bundle three files, unzip in test, assert byte identity.
- `lib/preview.test.ts` — image + PDF preview generation.

17.2 E2E (Playwright)

Run against a fully wired local stack (frontend + backend + LocalStack S3 + Postgres). Ten critical scenarios from §14.2 of the Master document.

17.3 Visual Regression

Playwright screenshot diffs for the upload, viewer, and success states in both light and dark themes.

18 Frontend Tickets

Frontend-owned tickets from the master ticket list, with concrete acceptance criteria.

`SS-011` React + Vite scaffold

TS strict mode, router for `/`, `/v/:fileId`, `/status/:fileId`, `/delete/:fileId`. ESLint security rules enabled.
AC: `pnpm dev` runs; `pnpm build` emits SRI hashes; `pnpm typecheck` clean.

`SS-012` Upload page

Idle / progress / success states.
AC: drag-drop, password input, encrypt flow visible, share link displayed, password cleared on unmount.

`SS-013` Viewer page

All 8 states from §10.4.
AC: tampered ciphertext shows error and offers no download.

SS-020-FE TTL segmented control

v2.0

F2.
AC: default 7d; live countdown on viewer.

SS-023-FE Status page

v2.0

F4.
AC: reads token from fragment, polls every 30s, never displays IP or recipient identifier.

SS-024-FE Access password verifier UI

v2.0

F5.
AC: wrong password shows error without network call; right password reveals encryption-password prompt.

SS-025 Link-only mode

v2.0

F6.
AC: mode selector toggles; key embedded in fragment; viewer skips password prompt.

SS-026 Multi-file / folder bundling

v2.0

F7.
AC: upload 3 files → recipient downloads ZIP containing 3 byte-identical files.

SS-027 In-memory preview

v2.0

F8.
AC: PNG previews; PDF page 1 previews; no `indexeddb` / `filesystem` writes (verified via Chrome devtools).

SS-028 QR popover

v2.0

F9.
AC: QR scans correctly on iOS / Android camera apps.

SS-029 How It Works modal

v2.0

F11.
AC: reachable from header; focus trap; animations respect reduced motion.

SS-030 Theme system

v2.0

F12.
AC: no FOUC on cold load; system / dark / light persisted; respects OS change live.

SS-031 UI library (12 components)

v2.0

All 12 components from §9.
AC: Storybook stories cover every documented state.

SS-034 Tampered blob handling

v2.0

AEAD failure routes the viewer to the Tampered state; no download button rendered.

SS-035 404 + Unsupported pages

v2.0

Both fully branded.
AC: unsupported page shows min-versions table.

SS-036 Accessibility pass

v2.0

WCAG 2.2 AA, Lighthouse a11y ≥ 95.

SS-037 Mobile breakpoints

v2.0

Bottom sheet under 640 px; trust badges wrap; touch targets ≥ 44 px.